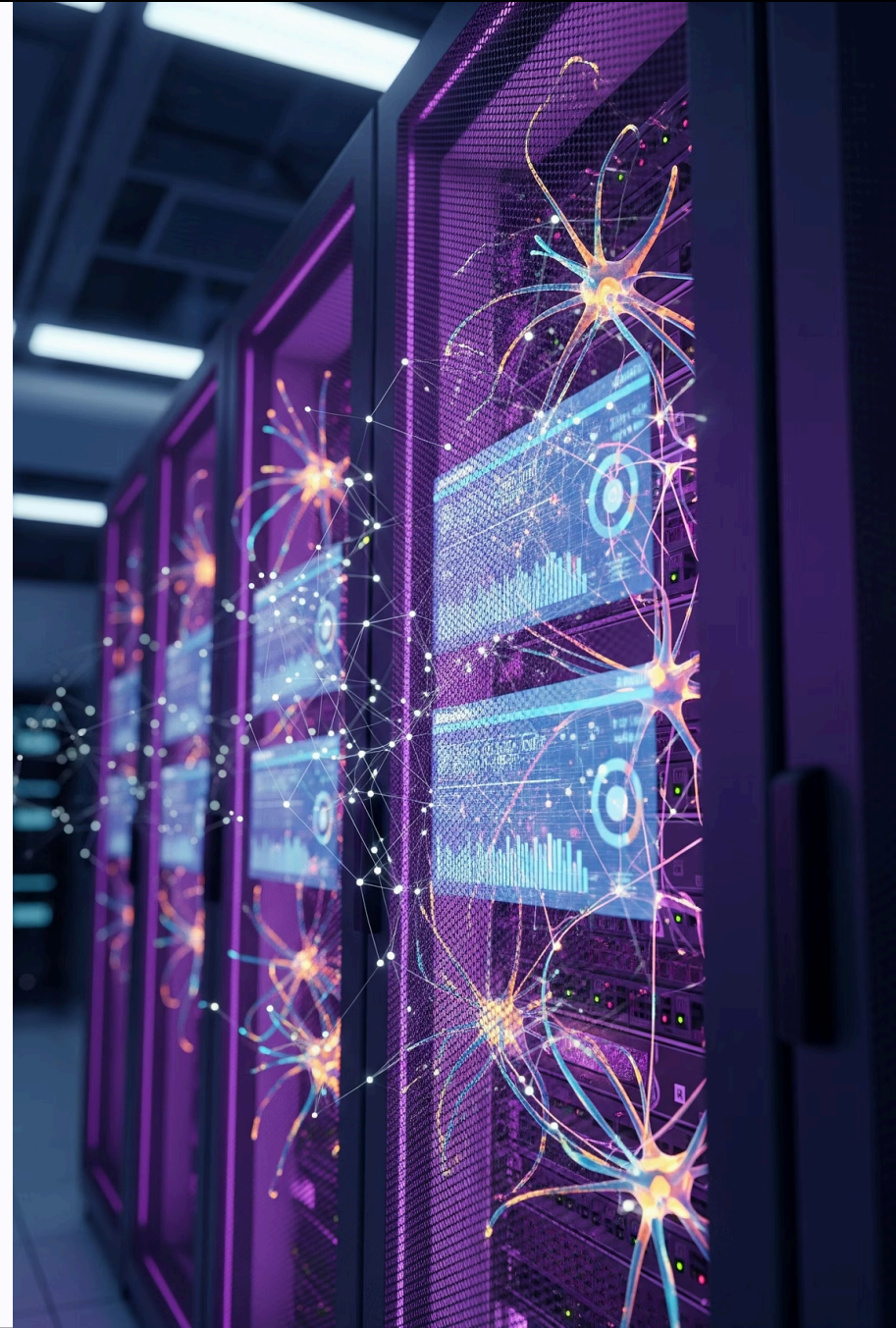


# Caching Strategies for LLMs

## Reducing Latency and Cost in Inference

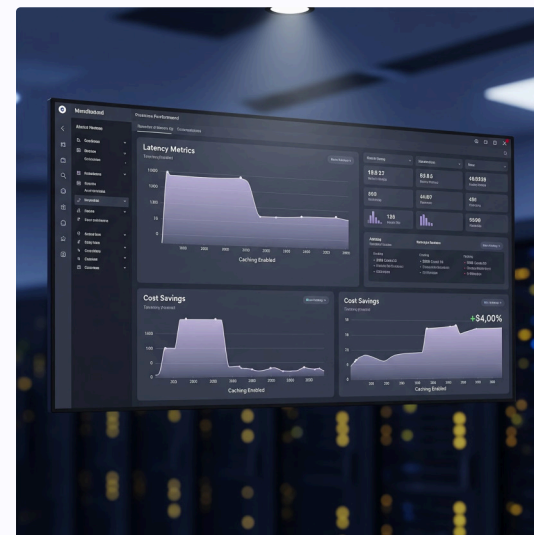
A technical guide for ML engineers and infrastructure architects looking to optimize their LLM-powered systems.



# Why Caching Matters

LLM inference is **expensive and slow**, creating bottlenecks in production environments. With many queries being repetitive or partially overlapping, caching provides a strategic advantage by:

- Eliminating redundant compute operations
- Dramatically improving **latency and throughput**
- Reducing operational costs by 30-70% in high-volume applications
- Enhancing user experience with faster response times



# Types of Caching Strategies for LLMs

1

## Prompt/Response Cache

Stores complete query → answer pairs for exact matching. Provides full response retrieval without computation when identical queries are encountered.

2

## Embedding Cache

Reuses vector representations for repeated documents or queries, avoiding costly re-embedding operations in retrieval pipelines.

3

## KV-Cache

Preserves attention key-value states between inference steps, drastically accelerating token generation in long contexts.

4

## Retrieval Cache

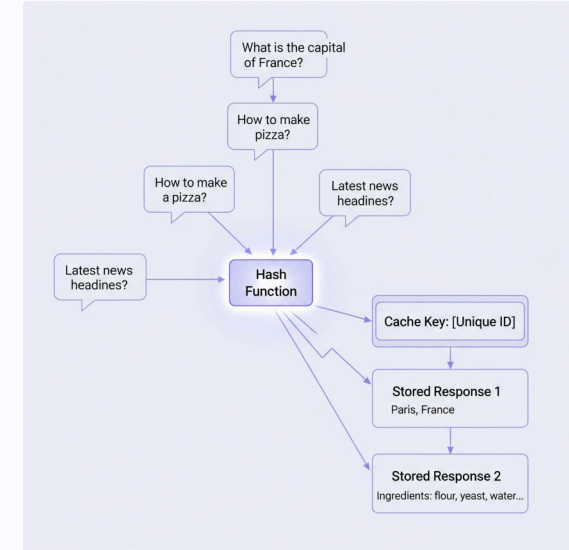
Stores RAG search results for frequent queries, bypassing expensive vector similarity searches on repeated information needs.

# Prompt/Response Caching

Prompt/response caching excels for frequently repeated queries in production systems, offering full response retrieval with zero inference latency.

## Implementation Approach

- Hash the entire query + model parameters as cache key
- Store full model responses as cache values
- Implement fuzzy matching for near-duplicate detection
- Set appropriate TTL based on content freshness needs



✓ **Ideal Use Case:** Customer support bots handling FAQs where many users ask identical questions about policies, products, or procedures.



# KV-Cache in Transformers

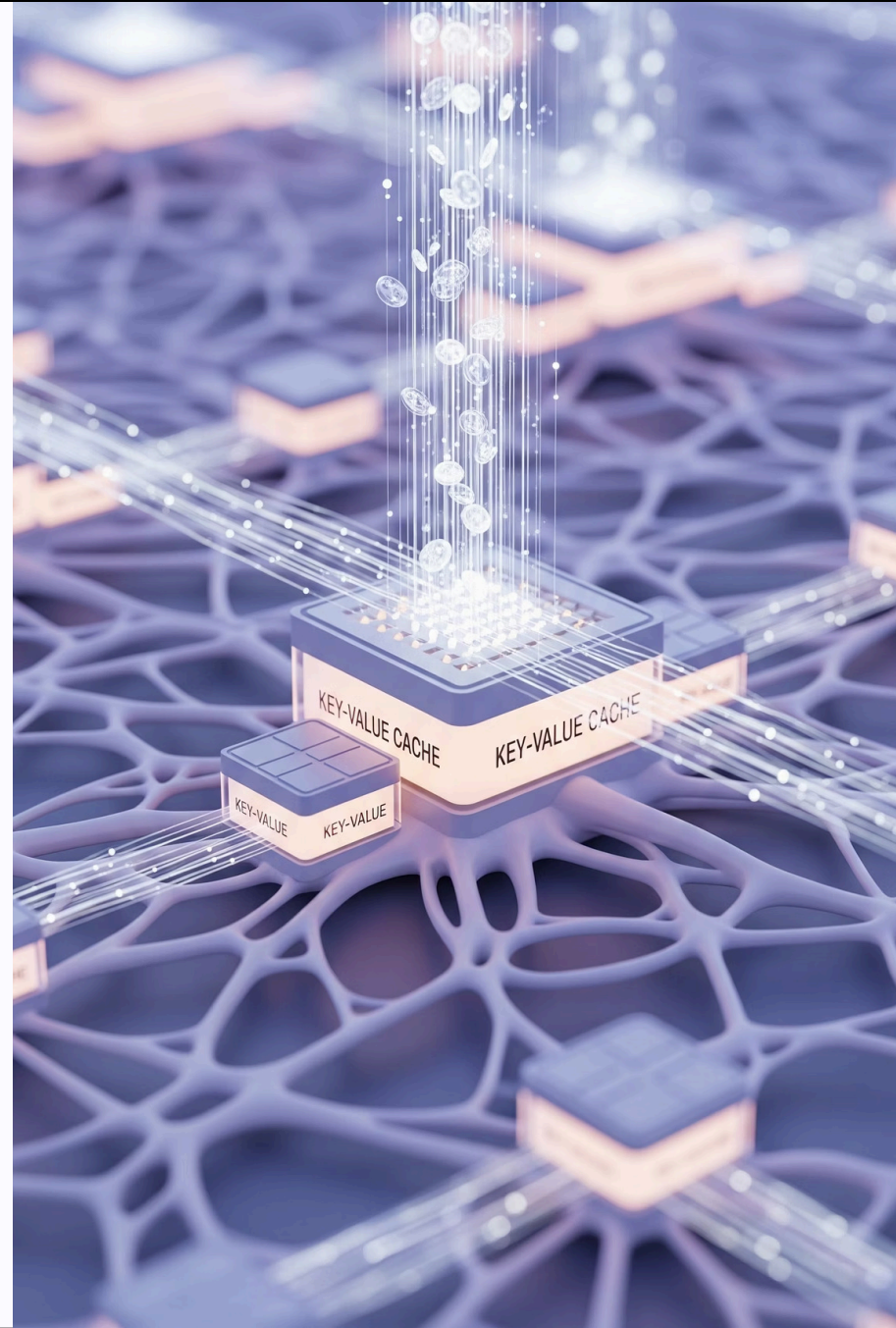
The KV-Cache stores intermediate attention states for each processed token, enabling transformers to avoid redundant computation during autoregressive decoding.

## Technical Benefits

- Reduces inference complexity from  $O(n^2)$  to  $O(n)$
- Cuts per-token generation time by 30-50%
- Essential for **long-context inference** (8K+ tokens)
- Enables real-time chat applications at scale

## Implementation Challenges

- Memory footprint grows linearly with context length
- Requires GPU memory management strategies
- Must balance between sequence batching and cache size
- Pruning techniques needed for ultra-long contexts



# Infrastructure Considerations

## Cache Store Options

- Redis: High-throughput, distributed caching
- Memcached: Simple, low-latency key-value store
- In-GPU memory: Fastest access but limited capacity
- Hybrid approaches: Tiered caching for different types

## Eviction Policies

- LRU: Removes least recently used entries
- LFU: Prioritizes frequently accessed items
- TTL-based: Expires entries after set duration
- Size-based: Caps memory usage with watermarks

## Consistency Management

- Model versioning in cache keys
- Flush strategy on model updates
- Canary deployments with cache warming
- Monitoring for stale response detection

The optimal caching infrastructure balances **memory footprint against performance gains**, requiring careful tuning based on your specific workload patterns and infrastructure constraints.

# AI/ML Use Cases



## Customer Service Chatbots

Cache common questions about products, policies, and troubleshooting to handle high-volume, repetitive inquiries without repeated inference.

## Search & RAG Applications

Cache document embeddings and frequent query results to eliminate redundant vector computations and similarity searches.

## Multi-turn Dialogue Systems

Leverage KV-caching to maintain conversation context efficiently, reducing token generation latency by up to 70% for fluid interactions.

## Content Moderation

Cache classification results for similar content patterns to accelerate high-throughput moderation workflows.

# Best Practices

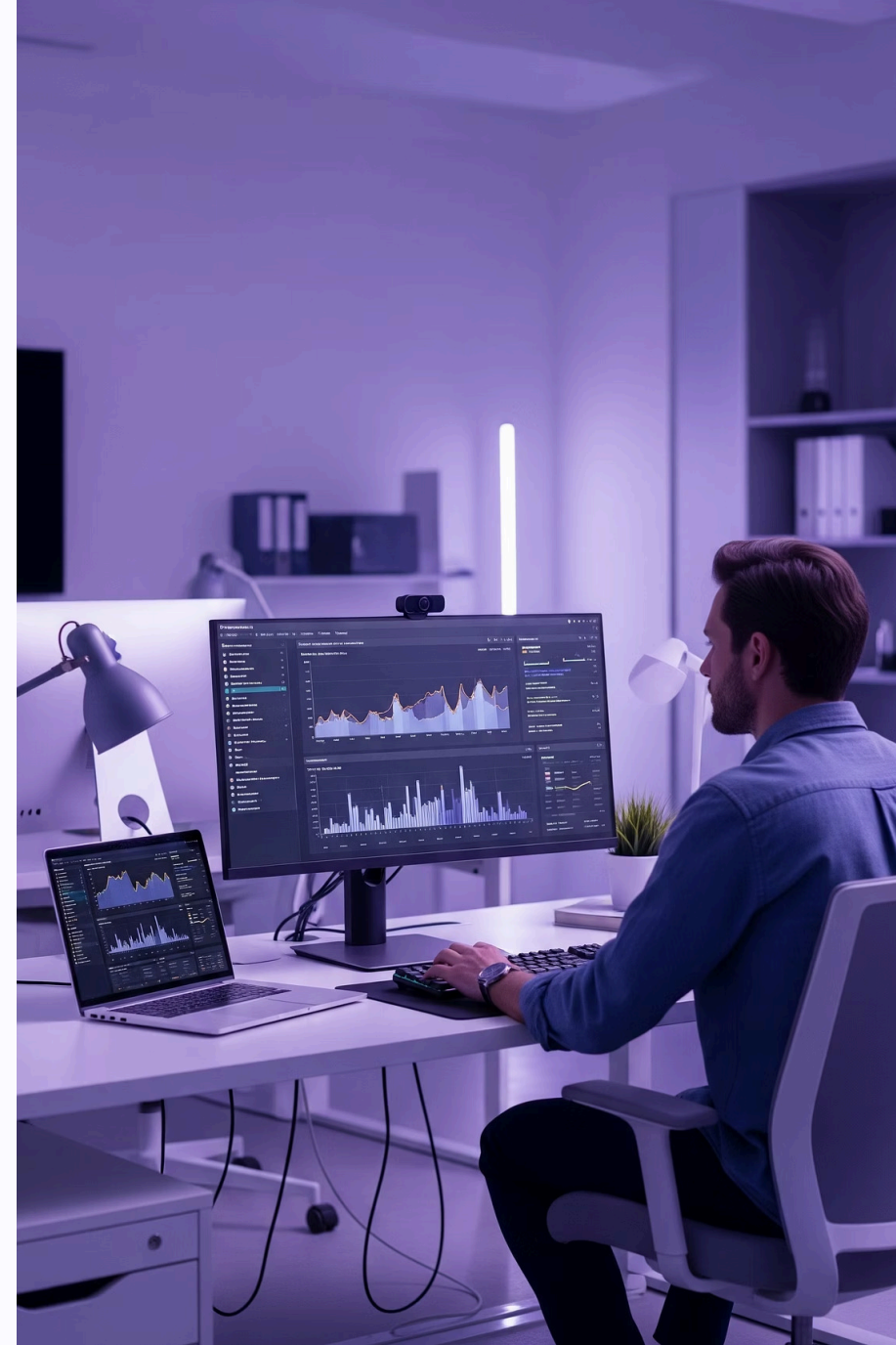
## Implementation Guidelines

- **Cache selectively** - Focus on high-value, repetitive queries
- **Monitor hit/miss ratios** - Target >40% hit rate for ROI
- **Implement cache warming** - Pre-populate with common queries
- **Version cache keys** - Include model ID/version in keys

## Optimization Strategies

- **Semantic deduplication** - Group similar queries
- **Adaptive TTL** - Longer expiry for stable content
- **Partial result caching** - Store intermediate steps
- **Probabilistic caching** - Focus on high-value entries

☐ Continuously balance between **cost savings** and **memory usage** by adjusting cache policies based on actual traffic patterns.





# Key Takeaways

70%

## Latency Reduction

Potential improvement from effective caching strategies in high-volume LLM applications

60%

## Cost Savings

Typical reduction in inference compute costs with properly implemented caching

4

## Cache Types

Complementary caching strategies to implement across your ML infrastructure

**Caching is the key to scaling LLM serving efficiently without compromising quality.**

Implement a layered approach combining prompt/response, embedding, KV, and retrieval caches. Start with KV-caching for immediate performance gains, then expand to other strategies based on your specific traffic patterns and budget constraints.